

Lab 4: VibeCoding – Lava Escape Repair Lab

Student: Suhas S Iyengar | **ID:** PES1UG24AM291 | **Date:** October 6, 2026

Course: Software Engineering Lab (SETAPESU26) | **Assigned Repo:** 52_lava-escape

Deliverable	Details / Location	Status
Before Gameplay Video	Screen Recording (10s before changes showing bug)	Completed
After Gameplay Video	Screen Recording (10s after all 4 tasks implemented)	Completed
Task 1 Commit	1bc1a42 - Fix platform underside collision snapping bug	Committed
Task 2 Commit	9f4aefc - Implement crumbling platform hazards	Committed
Task 3 Commit	95446d9 - Implement high-velocity spring platforms	Committed
Task 4 Commit	f15f119 - Implement rising danger HUD & lava burst surges	Committed
Target Submission Repo	se_lab1 / lab4 / (code, videos, chat export)	Organized

1. Initial Assessment & 'Before' Gameplay Recording

Prompt / Action: User requested to run the initial code to play and record the 10-second 'before' video.

Observed Bug: In game/player.py, collision condition checked `self.rect.bottom <= p.bottom + 10` whenever `vel_y > 0`. When jumping upward from beneath a ledge, reaching the jump apex or descending through the underside caused the player's feet to prematurely snap to the top ledge surface instead of passing through cleanly.

Recording Captured: Screen Recording 2026-10-06 141251.mp4 (saved as before_recording.mp4).

2. Task 1: Fix Platform Underside Collision Snapping Bug

Objective: Ensure landing resolution only triggers when the player is descending and strictly clears the platform's upper edge.

User Prompt:

"Fix the platform collision snapping bug in game/player.py. Ensure that when jumping from underneath, the player does not snap to the platform top unless they have strictly cleared the upper edge."

Resolution Applied: Tracked `prev_bottom = self.rect.bottom` before adding vertical velocity. Collision and landing resolution now strictly require `prev_bottom <= plat_rect.top + 4` in addition to downward motion (`vel_y >= 0`). This completely prevents snapping from beneath while preserving responsive ground detection.

Verification: Headless automated unit test verified jumping from below passed cleanly through without snapping, while landing from above grounded securely. **Commit:** 1bc1a42.

3. Task 2: Implement Crumbling Platform Hazards

Objective: Introduce fragile platforms that shake and disintegrate shortly after the player lands on them.

User Prompt:

"Implement crumbling platform hazards in game/world.py. Add an OOP Platform class where crumbling platforms begin shaking and collapse into falling debris after being stepped on, forcing constant upward mobility."

Resolution Applied: Replaced raw pygame.Rect in game/world.py with an object-oriented Platform(pygame.Rect) class. When stepped on, fragile platforms enter a 50-frame crumbling phase with escalating horizontal jitter (shake_offset_x), crack rendering, and flash tint. Upon timer expiration, is_broken = True is set and 14 falling rubble debris particles are spawned. The player immediately falls through.

Verification: Automated test confirmed stepped_on triggers upon landing, platform breaks after 50 frames, and subsequent updates allow player to fall through freely. **Commit:** 9f4aefc.

4. Task 3: Implement High-Velocity Spring Platforms

Objective: Add distinct high-powered spring platforms that launch the player upward with bonus vertical velocity when touched.

User Prompt:

"Implement high-velocity spring platforms. Add distinct green spring platforms that launch the player with bonus vertical velocity (e.g. -20 vs standard -13) with a bounce animation and launch sparkle particles."

Resolution Applied: Added kind='spring' to Platform with custom emerald-green styling, coiled chevron indicators, and metallic base. Upon player contact, trigger_spring() instantaneously propels the avatar with vel_y = -20 (reaching >360px vertical height), compresses the spring pad for 12 frames, and ejects a fan of bright launch spark particles.

Verification: Test verified player.vel_y == -20 immediately upon contact, spring compression activated, and particles spawned. **Commit:** 95446d9.

5. Task 4: Implement Rising Danger HUD & Lava Burst Surges

Objective: Implement a danger meter tracking the speed of rising lava paired with periodic lava burst surges.

User Prompt:

"Implement a Rising Danger HUD & periodic Lava Burst Surges in game_engine.py and world.py. Track lava speed with a color-coded danger gauge and trigger timed lava surges with warning banners and fire burst particles."

Resolution Applied:

- **Surge Phase Cycle:** Introduced a 540-frame (~9s) recurring cycle: normal ascent (0-380 frames), surge_warning with amber pulsing alert (380-450 frames), and surge_active with 2.6x speed acceleration (450-540 frames).
- **Lava Burst Aesthetics:** Enhanced draw_lava with higher wave amplitude (14px), faster oscillation (0.22 speed), radiant crest lines, and incandescent fire spark particles spraying from lava crests.
- **Danger Meter HUD:** Renders at top-right with an interactive fill bar dynamically transitioning Green (Low) -> Yellow (Med) -> Orange (High) -> Pulsing Crimson (Surge Burst!), alongside exact numerical speed readouts and flashing center banners.

Verification: Simulation verified warning phase trigger at frame 385, active surge acceleration at frame 460, and zero-error HUD rendering. **Commit:** f15f119.

6. 'After' Gameplay Video & Deliverables Submission

- Captured `after_recording.mp4` demonstrating clean one-way jumps, crumbling platforms, spring rocket launches, and active lava surges with the Danger HUD.
- Organized all lab deliverables into `se_lab1/lab4/` containing the complete updated code, both mp4 recordings, and this exported documentation.